Language models from scratch

Smitha Soman | CMPE 258 | HW 1 - Part 1

For this assignment, I compared how well recurrent models and Transformers predict the next character in Tiny Shakespeare. I gave each model the same number of training tokens so that a model would not have an advantage simply from seeing more text. I also checked what the evaluation metrics mean, varied the context length, and looked at attention patterns in a trained Transformer.

The dataset has 1,003,854 training tokens and 111,540 validation tokens, with 65 characters in the vocabulary. Each architecture was trained for 2,000 steps, or 16,384,000 tokens, using seeds 42 and 1337. I ran the experiments on a Tesla T4 in Colab. I mainly compare validation bits per byte (bpb), where a lower value means better prediction.

Checking the metrics by hand
The saved loss, L, is already the mean negative log-likelihood in natural-log units. If N is the number of validation tokens and B is the number of bytes, the conversions are:

Total negative log-likelihood = $L \times N$
Perplexity = $\exp(L)$
Bits per token = $L / \ln(2)$
Bits per byte = $L \times N / [B \times \ln(2)]$

I used the modern Transformer run with seed 1337. Its metrics.json gives L = 1.4607713136221696 and N = B = 111,360. Multiplying L by N gives 162,671.493485 nats; dividing this total by N recovers the logged loss. This checks the saved values and their units, rather than independently recalculating loss from the model's predictions.

Metric	My calculation	Logged value
Perplexity	4.309282055	4.309282055
Bits per token	2.107447530	-
Bits per byte	2.107447530	2.107447530

Before training
For the untrained checks, I ran one step with learning rates and weight decay set to zero, so the weights stayed at initialization. A uniform predictor assigns probability 1/V to every token, giving loss ln(V) and perplexity V. Both evaluations scored 111,488 tokens and bytes.

Tokenizer	V	ln(V)	Measured loss	Perplexity	bpb
Character	65	4.174387	4.162828	64.2530	6.0057
Byte	256	5.545177	5.559498	259.6926	8.0207

The losses differ from ln(V) by only -0.011559 for characters and +0.014321 for bytes. Initialized predictions are not exactly uniform, so a small difference is expected. The byte model's perplexity can exceed 256: vocabulary size is the uniform reference, not a maximum possible perplexity.

Comparing the architectures

The modern Transformer finished with the lowest validation bpb. Among the recurrent models, LSTM-256 performed best. It was also much quicker to train: about one minute per run, compared with three minutes for the modern Transformer. The table reports the mean and sample standard deviation across the two seeds.

Model	Parameters	Validation bpb ± SD	Mean time
Vanilla RNN	280,320	2.4392 ± 0.0041	0.264 min
GRU	806,656	2.1595 ± 0.0009	0.779 min
LSTM-256	1,069,824	2.1389 ± 0.0034	1.020 min
LSTM-512	4,236,800	2.1754 ± 0.0100	3.475 min
Textbook GPT	3,438,336	2.1837 ± 0.0059	2.617 min
Modern GPT	3,162,624	2.1065 ± 0.0013	3.023 min

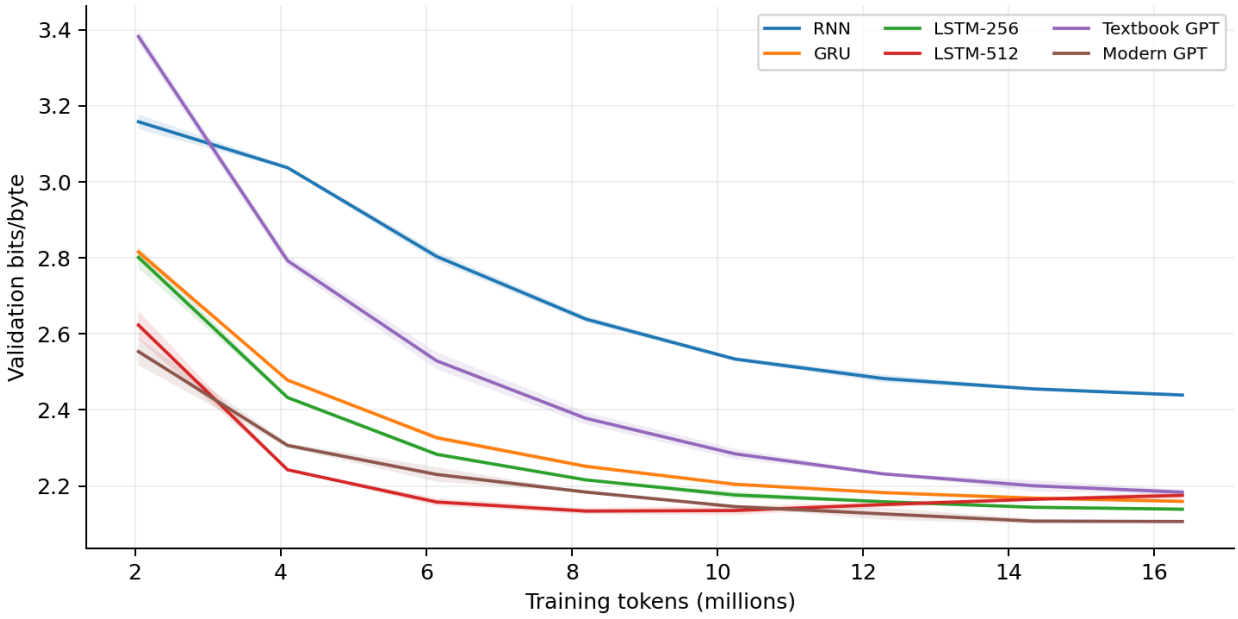


Figure 1. Validation bpb versus training tokens. Each line averages the two seeds; shading shows ± one sample standard deviation. All runs end at 16,384,000 tokens.

The vanilla RNN is clearly behind in the later part of the learning curves. An RNN has to carry information forward through repeated hidden-state updates. Gradients can shrink or grow over these steps, which makes long dependencies difficult to learn. The gates in LSTMs and GRUs give them more control over what to retain and forget. That helps explain the pattern, although I did not measure gradients directly.

Capacity is another difference: the RNN has only 280,320 parameters, fewer than any other model here. The recurrent models and Transformers also use different learning rates and weight decay. These runs therefore compare the supplied configurations; they do not isolate the effect of architecture alone.

The same token budget also does not mean the same compute cost. Using the rough estimate 6ND, where N is total parameters and D is training tokens, one run ranges from 0.028 PFLOPs for the RNN to 0.416 PFLOPs for LSTM-512. This estimate leaves out architecture-specific costs, including attention, so the recorded training times are useful alongside it.

The larger LSTM overfits

Increasing the LSTM hidden size from 256 to 512 did not improve its final validation result. The larger model has almost four times as many parameters, but its mean bpb is 2.1754, compared with 2.1389 for the smaller model. The curves explain why.

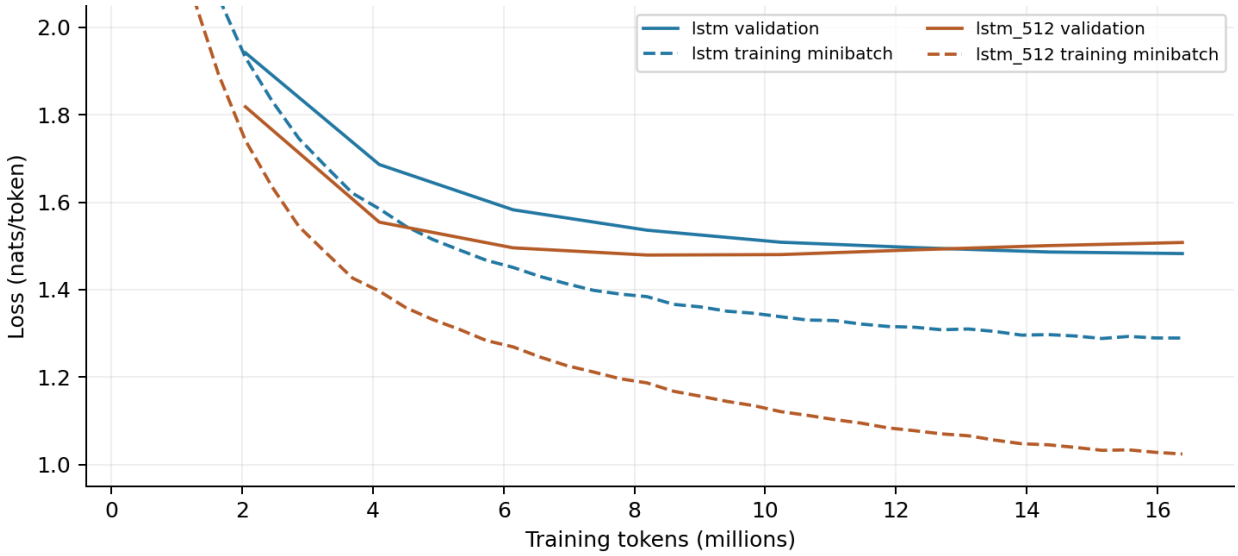


Figure 2. Training-minibatch and validation loss, averaged across seeds. Training loss is measured with dropout active, while validation uses evaluation mode, so their absolute gap should be interpreted cautiously.

For the larger LSTM, training loss keeps falling after validation loss has stopped improving. Its best recorded validation scores occur at step 1,250 for seed 42 and step 1,000 for seed 1337. Both runs get worse by the final step.

LSTM-512 seed	Best evaluated step	Best bpb	Final bpb
42	1,250	2.1272	2.1683
1337	1,000	2.1398	2.1825

The final training losses are about 1.02-1.03 nats for the large model and 1.28-1.30 for the small one. Together with the rising validation loss, this supports overfitting: the extra capacity helps the model fit the training text, but that improvement does not carry over to validation. Early stopping or stronger regularization might help, though I did not test either.

The modern Transformer beats LSTM-256 by 0.0324 bpb on average, while the RNN is worse by 0.3003. The saved analysis includes paired, two-sided t-tests, with unadjusted p-values of 0.0280 and 0.0011 for those comparisons. The large-LSTM comparison gives $p = 0.0815$. These tests use only two seed pairs and five comparisons against the LSTM, so I treat them as exploratory rather than strong evidence about how every future run would behave.

The median within-model standard deviation is 0.0037 bpb. That gives some sense of run-to-run variation, but twice this value is not a formal significance cutoff. More seeds would be needed to judge the smaller differences confidently.

Increasing the context length

I trained LSTM-256 and the modern Transformer with context lengths of 64, 128, 256 and 512 characters. To keep the token budget fixed, I used batch sizes of 128, 64, 32 and 16, respectively. Each run still had 8,192 tokens per step and 2,000 steps. I reused the four existing 256-context runs and trained the other twelve.

Context	Modern GPT bpb ± SD	LSTM bpb ± SD
64	2.14719 ± 0.00276	2.18496 ± 0.00297
128	2.11004 ± 0.00738	2.16106 ± 0.00040
256	2.10650 ± 0.00134	2.13887 ± 0.00336
512	2.09384 ± 0.00387	2.12802 ± 0.00008

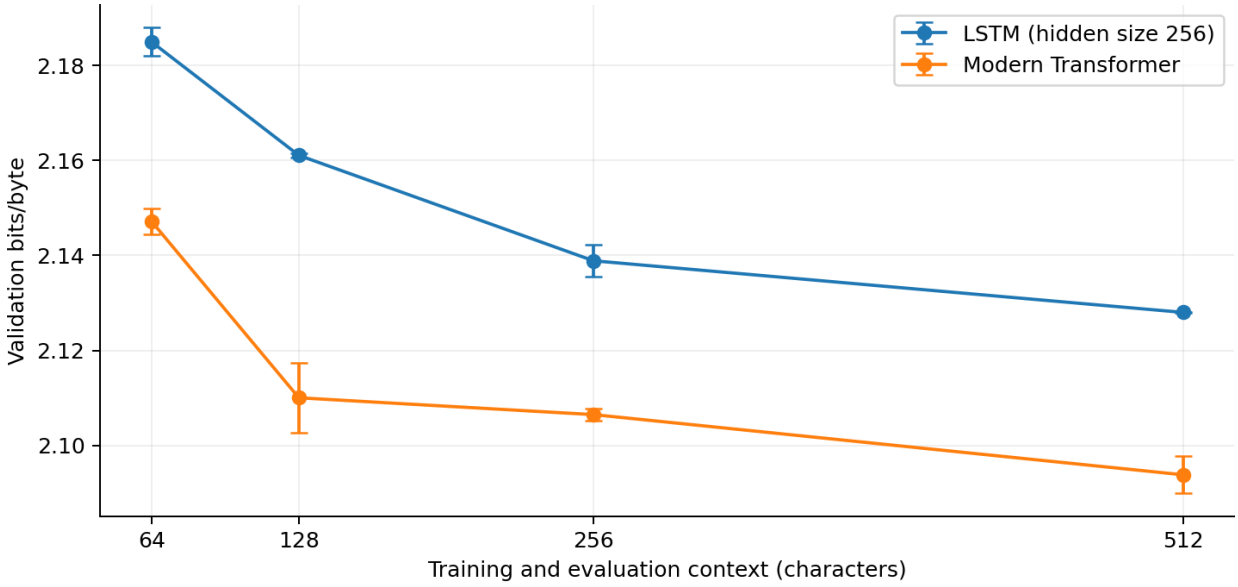


Figure 3. Validation bpb at each context length. Error bars show ± one sample standard deviation across seeds. Both training and evaluation context change in this experiment.

Neither model reaches a clear stopping point in the tested range. Both have their lowest mean bpb at 512 characters. The LSTM improves by 0.02390, 0.02219 and 0.01086 bpb as the context doubles, so its gains become smaller. The Transformer improves by 0.03714, 0.00354 and 0.01266. Its curve is nearly flat from 128 to 256, but improves again at 512; I would not call the earlier flat section a plateau.

The Transformer has a better score at every length, but that does not show that it gains more from extra context. From 64 to 512 characters, the total improvement is 0.05335 bpb for the Transformer and 0.05694 for the LSTM. The gains are similar, with a slightly larger numerical improvement for the LSTM.

There are two limits to this comparison. First, training and evaluation lengths change together, so it does not measure the benefit of longer context at inference alone. Second, the scored token counts are 111,488, 111,488, 111,360 and 111,104 at the four lengths. The validation targets therefore differ slightly, and two seeds are too few to put much weight on small changes.

Looking at attention

I used the modern Transformer checkpoint from seed 1337 to inspect three texts: dialogue ("ROMEO:\nBut soft! what light through yonder window breaks?", with \n representing a newline), prose ("The little bird flew over the garden and into the trees."), and repetition ("the cat the cat the cat the dog the dog the dog"). The layer and head numbers below start at zero.

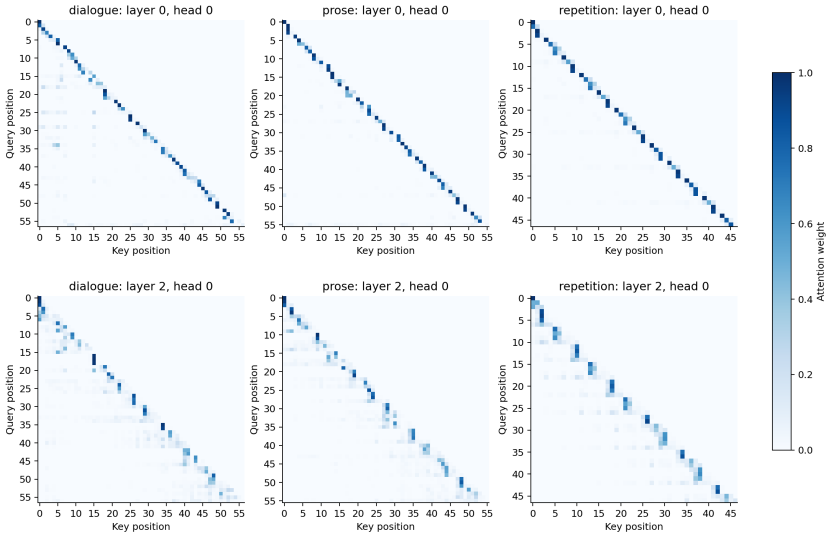


Figure 4. Layer 0, head 0 (top) and layer 2, head 0 (bottom), shown for all three texts. Future positions have zero attention because of the causal mask.

Head and target	Dialogue	Prose	Repetition	Text mean
L0 H0: previous character	0.4978	0.5915	0.6517	0.5803
Uniform reference	0.0648	0.0657	0.0747	0.0684
L2 H0: word start*	0.2399	0.2521	0.1840	0.2253
L2 H0: previous character*	0.1084	0.1050	0.0451	0.0861
Uniform reference*	0.0587	0.0622	0.0748	0.0652

Layer 0, head 0 puts an average weight of 0.5803 on the previous character, well above the uniform reference of 0.0684. I calculated this by averaging A[t,t-1] over positions $t \geq 1$, then averaging the three text means equally.

Layer 2, head 0 favors the first character of the current word. For this check (*), I included only third and later letters in alphabetic words. Otherwise, the word start could also be the previous character. On these rows, mean word-start attention is 0.2253, compared with 0.0861 on the previous character and a uniform reference of 0.0652. The reference averages 1/(t+1) over the same rows.

These measurements support two attention preferences, but the heads were selected after inspection and tested on only three short texts. They do not establish a dedicated linguistic function or show how much each head contributes to prediction.

What I take from the experiments

The modern Transformer gave the best final score, while the smaller LSTM was a good option when training time mattered. More parameters did not automatically help: the larger LSTM overfit. Longer context helped both models on average through 512 characters, but this experiment did not show a greater context benefit for the Transformer. These findings cover only two seeds and one dataset.

Appendix: settings and reproducibility

Training settings. The recurrent models have two layers and hidden size 256, except for LSTM-512. Their learning rate is 0.002 and weight decay is zero. The Transformers have four layers, dimension 256 and four heads, with learning rate 0.001 and weight decay 0.1. Architecture runs use sequence length 256, batch size 32 and gradient accumulation 1.

All configurations use dropout 0.1, AdamW with betas (0.9, 0.95), a cosine schedule, 100 warmup steps and gradient clipping at 1.0. Textbook GPT uses LayerNorm, learned positions, GELU and biases. Modern GPT uses RMSNorm, rotary positions, SwiGLU and no biases. Maximum model context is 1,024. The saved configurations disable compilation; the device display can show compilation capability even when compilation is not enabled.

Validation is logged every 250 steps. With final_eval_tokens set to zero, final evaluation uses the available validation windows. The context sweep keeps the model settings and training schedules unchanged while adjusting batch size inversely with sequence length.

Environment and versions. The Colab environment reported Python 3.13.16, PyTorch 2.11.0+cu130 and a Tesla T4. Training used CUDA and FP32. Architecture runs record commit cc64ca3ed3aa1ce7df3cf381c492bc7a9826b749; later runs record 8418fc1bae85657190f11749483bf2e729b9702c. The diff check found no changes within DeepDataMiningLearning/llm/nanolm between those commits. Both versions are recorded in the supporting files.

Compute. There are 26 unique saved runs: 12 architecture runs, 12 additional context runs and two untrained checks. Their logged training times total about 46.72 minutes. This is not the total time spent on the assignment: it excludes setup, benchmarks, analysis, interruptions and overhead outside the training timer. The approximate total from 6ND is 5.051 PFLOPs, counting reused runs only once.

Checks and setup issues. The recomputed metrics match the logged values. Attention rows sum to one within about 2.4×10^{-7}, and weights on future positions are zero. The initial test suite had 29 passes, one skip and one DPO/LoRA failure from an incompatible torchao version. That test concerns optional fine-tuning dependencies; I do not count it as a passing test. For the untrained checks, I used one zero-update step because the scheduling code does not support zero training steps.

Supporting files. The repository contains metrics, logs, original and portable configurations, summary tables, attention arrays, figures and analysis scripts. Portable configurations preserve the numerical settings while changing output paths. Checkpoints and the dataset are not included. The gate-by-gate GRU extra-credit check is documented in the next appendix.

Code and assignment source. Training uses the provided nanolm implementation in DeepDataMiningLearning. The analysis scripts use the saved outputs to generate the report figures and metric checks.
Course tutorial: TUTORIAL_nanolm.md
Project repository: SmithaSJSU/language-models-from-scratch

Appendix: GRU extra credit

For extra credit, I implemented a GRU cell gate by gate and compared it with PyTorch nn.GRU. This is a numerical check of the recurrent calculations, so no language-model retraining was needed.

At each time step, the reset gate controls how much of the transformed previous state enters the candidate. The update gate then controls how much of the old state is retained. Using elementwise multiplication, the calculations are:

r = sigmoid(W_ir x + b_ir + W_hr h_prev + b_hr)
z = sigmoid(W_iz x + b_iz + W_hz h_prev + b_hz)
n = tanh(W_in x + b_in + r × (W_hn h_prev + b_hn))
h = (1 - z) × n + z × h_prev

The reset gate placement matters. To match PyTorch, r multiplies the hidden-state projection after its matrix multiplication, including the hidden bias. Multiplying the previous state by r before the matrix multiplication would implement a different variant. I split the packed parameters into reset, update and candidate sections, then computed each gate explicitly.

For verification, I copied all weights and biases from a single-layer nn.GRU into the scratch cell. Both versions received the same input and initial hidden state. I looped the scratch cell over the sequence and compared every output as well as the final hidden state.

Test setting	Value
Environment	Colab; PyTorch 2.14.1+cu130; CPU; float64
Dimensions	Batch 3; input size 5; hidden size 8
Seeds	42, 1337, 2026
Sequence lengths	1, 7, 32
Initial hidden state	Zero and random
Reference configuration	One layer; unidirectional; biases enabled
Assertion tolerance	Absolute 1e-12; relative 1e-10

Measured result	Value
Cases passed	18 / 18
Largest sequence-output difference	3.331×10^{-16}
Largest final-hidden-state difference	2.220×10^{-16}

All 18 cases passed. The largest differences are consistent with rounding in double precision. This supports numerical agreement for the configurations tested; it does not establish agreement for all possible inputs or verify gradients, multiple layers, bidirectional operation or GPU execution.

The source and verification script are in extra_credit/gru_scratch.py and extra_credit/verify_gru.py. The accompanying CSV records the displayed Colab results. This check ran in a newer PyTorch environment than the earlier training experiments, so I recorded its version separately.

Reference. PyTorch GRU documentation, including gate equations and reset-gate convention.